
BUDT 758T · Data Mining and Predictive Analytics
Robert H. Smith School of Business · University of Maryland
Spring 2026

Final Project Report

Predicting Highly Useful Yelp Reviews

GROUP 1

Komal Abid
Kunal Roy
Shikha Sharma
Kate McKee
Roshiv Giri

Section 1: Team Member Names and Contributions

Team Member	Primary Role	Contributions
Komal Abid	Project Lead & ML Engineer	Directed the overall modeling strategy. Led LightGBM hyperparameter tuning, cross-validation design, VADER integration, final model training, and prediction generation. Wrote Sections 2, 4, and 5 of the report.
Kunal Roy	Data Engineer & Infrastructure	Set up the Kaggle environment and dataset hosting. Engineered supplementary dataset features (business, user, tip). Managed memory optimization and pipeline stability. Wrote Section 3 feature table.
Shikha Sharma	Feature Engineering & EDA	Designed and implemented the 56 structured features across all feature groups. Produced all 10 EDA charts with interpretations. Wrote Section 3 data understanding narrative.
Kate McKee	Model Comparison & Evaluation	Trained and evaluated Logistic Regression, Decision Tree, Random Forest, and HistGradientBoosting models. Implemented the best_threshold_at_fpr evaluation function. Drafted Section 4 model comparison tables.
Roshiv Giri	Text Features & Ablation	Built the TF-IDF pipeline and conducted both ablation studies (text features and external VADER data). Wrote the ablation evidence narrative and external data justification for the report.

Section 2: Business Understanding

Executive Summary

Yelp hosts over 200 million reviews of local businesses, from restaurants and cafes to car mechanics and hair salons. The platform's core value proposition rests on the assumption that the most helpful reviews surface naturally through community voting. In practice, however, this mechanism is slow: genuinely useful reviews can languish unread for months before accumulating enough votes to gain visibility. Only 12.12% of reviews in this dataset ever achieve top_useful status, meaning the vast majority of high-quality content never reaches its potential audience.

Our model directly addresses this problem. By predicting whether a newly submitted review will become highly useful - using only information available at submission time - we enable the platform to proactively surface high-quality content, improve the reviewer experience, and generate measurable business value for all stakeholders.

Who Can Use This Model

The output of our model - a binary flag indicating predicted high usefulness - is a versatile decision-making input for several distinct stakeholder groups:

- **Yelp's product and platform team:** The model can be integrated into Yelp's content-ranking algorithm to surface predicted-useful reviews more prominently in search results and business pages before community voting converges. This reduces the discovery lag for high-quality content from weeks or months to near-instantaneous.
- **Business owners and operators:** A business intelligence dashboard built on this model could alert owners when a highly useful review is predicted for their location, enabling faster response, personalized engagement, and proactive reputation management at scale.
- **Content quality and trust teams:** Predicted usefulness scores can be used to prioritize human editorial review, flag spam or low-effort content, and identify patterns in what makes reviews genuinely helpful, informing platform-wide quality guidelines.
- **Yelp Elite program management:** Feature importances confirm that user elite status, prior useful votes, and fan count are among the three strongest predictors of review usefulness. These signals can inform Elite program admission criteria and reviewer development programs.
- **Advertisers and brand partners:** Placements positioned near predicted-useful reviews command higher engagement and therefore higher click-through value, creating a premium inventory tier based on predicted content quality rather than just page traffic.

Business Actions Enabled

Unlike many prediction problems where the model output is an input to a human decision, the binary nature of our predictions enables fully automated, scalable business actions:

- Automatically feature predicted-useful reviews in personalized email digests and push notifications to nearby users, increasing both review reach and user retention.
- Create a 'Fresh & Helpful' content module on business pages showing newly submitted predicted-useful reviews before they have accumulated votes, improving page engagement.
- Trigger automated 'thank you' messages to reviewers whose submissions are predicted as highly useful, improving reviewer satisfaction and platform loyalty.
- Use the 17.35% positive prediction rate on the test set (versus 12.12% historical rate) as an input to content strategy: the model identifies approximately 4,200 additional reviews per 100,000 submissions that merit elevated visibility.

Why This Metric - and What the Model Is Worth

The contest metric - maximum TPR at $\text{FPR} \leq 10\%$ - reflects the real production cost structure of this problem. Surfacing a review incorrectly (false positive) erodes user trust in the platform's recommendation quality. Missing a genuinely useful review (false negative) wastes a high-quality submission and discourages the reviewer. The 10% FPR constraint ensures automated surfacing remains trustworthy at scale.

Quantified value: In the training period, there are 191,147 highly useful reviews among 1,577,278 total. Our model's validation TPR of 0.6803 means approximately 130,000 of these reviews would be correctly identified and elevated. With a positive prediction rate of 17.35% (versus the 12.12% base rate), the precision of reviews flagged

by the model is approximately 52%, more than four times better than random. At Yelp's scale, this translates to millions of high-quality reviews reaching their intended audiences earlier and with greater visibility, directly improving both the consumer experience and the platform's core value proposition.

Section 3: Data Understanding and Data Preparation

3.1 Dataset Overview

File	Rows	Cols	Description
review_train_x.csv	1,577,278	6	Training review features: review_id, user_id, business_id, stars, text, date
review_train_y.csv	1,577,278	1	Training labels: top_useful (binary 0/1). Positive class = 12.12% (191,147 reviews)
review_test_x.csv	700,000	6	Test set - no labels provided. 700,000 predictions submitted as group_1_yelp.csv
business.csv	15,000	13	Business attributes including stars, review_count, hours, is_open, and categories
user.csv	992,656	8	User metadata: elite history, fans, useful votes, review count, and friends list
tip.csv	300,865	5	Short tips per business - used to compute tip_count as a business engagement proxy

3.2 Key Data Insights

- **Class imbalance:** 12.12% positive rate (191,147 top_useful = 1). Zero duplicate reviews in training data. The severe imbalance drives our use of scale_pos_weight = 7.3 in gradient boosting models, stratified cross-validation folds, and threshold optimization focused on TPR@FPR≤10% rather than accuracy.
- **Text length is a strong signal:** Highly useful reviews average 992.9 characters and 182.8 words versus 506.6 characters and 93.4 words for non-useful reviews - nearly twice as long. This directly justifies the inclusion of all text-length features and TF-IDF representations.
- **User reputation matters most:** 7.2% of users (71,540 of 992,656) have held elite status. Median friend count is 3. User-level features including elite_count, useful vote history, and fan count proved to be among the three strongest predictors of review usefulness in the final model.
- **Missing values handled cleanly:** business.hours: 5.17% missing to weekly_hours = 0. user.elite: 92.79% missing to elite_count = 0. user.friends: 38.61% missing to friends_count = 0. Two rows in training data had missing useful/review_count/fans from unmatched user joins - filled with 0. All missing value handling performed without reference to the target variable.

3.3 Feature Table - All 56 Engineered Structured Features

TF-IDF columns (300 features, Section 5) are excluded from this count. All features below are applied identically to train and test. Code references indicate Python notebook cell numbers (In[N]).

ID	Feature Name	Brief Description	Code Location
-	REVIEW METADATA (6 features)		In[13,15]
1	stars	Review star rating (encoded 1.04–10.00)	In[13]
2	review_length	Total character count of the review text	In[15]
3	word_count	Total number of words in the review	In[15]
4	avg_word_len	Mean character length of words (vocabulary complexity)	In[15]
5	punct_count	Count of punctuation marks: . , ! ; :	In[15]
6	caps_ratio	Proportion of uppercase characters (emphasis signal)	In[15]
-	TEMPORAL FEATURES (6 features)		In[16,20]
7	review_year	Year the review was written	In[16]
8	review_month	Month (1–12) - seasonal patterns in review volume	In[16]
9	review_dayofweek	Day of week (0=Monday, 6=Sunday)	In[16]
10	review_age_days	Days elapsed since review (reference: 2022-01-01)	In[16]
11	review_is_weekend	Binary: 1 if review posted on Saturday or Sunday	In[20]
12	review_quarter	Quarter of year (1-4)	In[20]
-	SENTIMENT & KEYWORD PROXIES (5 features)		In[17]
13	text_exclamation	Count of exclamation marks (enthusiasm signal)	In[17]
14	text_question	Count of question marks (uncertainty or curiosity)	In[17]
15	sentence_count	Number of sentences (review structural complexity)	In[17]
16	unique_word_ratio	Unique words / total words (vocabulary richness)	In[17]

ID	Feature Name	Brief Description	Code Location
17	digit_count	Count of numeric characters (specificity signal)	In[17]
-	EXTENDED TEXT READABILITY (5 features)		In[20]
18	stopword_ratio	Proportion of English stopwords (content density proxy)	In[20]
19	uppercase_word_count	Count of fully uppercase words (emphasis)	In[20]
20	avg_sentence_length	Mean words per sentence (writing complexity)	In[20]
21	punctuation_ratio	Punctuation count / review length	In[20]
22	exclamation_ratio	Exclamation marks / sentence count	In[20]
-	RAW USER FEATURES (6 features)		In[11,12]
23	elite_count	Number of years the user has held elite status	In[11]
24	friends_count	Number of friends on Yelp (network engagement)	In[11]
25	review_count	Total reviews written by the user (experience)	In[12]
26	useful	Total useful votes the user has received historically	In[12]
27	fans	Total fans the user has on Yelp	In[12]
28	average_stars	User's mean star rating across all reviews	In[12]
-	RAW BUSINESS FEATURES (4 features)		In[9,10,12]
29	business_stars	Business average star rating on Yelp	In[12]
30	business_review_count	Total number of reviews for the business	In[12]
31	weekly_hours	Total weekly operating hours (parsed from hours field)	In[9]
32	tip_count	Number of Yelp tips for the business (engagement proxy)	In[10]
-	BUSINESS & OPINION FEATURES (4 features)		In[19,20]
33	business_is_open	Binary: 1 if business is currently open	In[19]

ID	Feature Name	Brief Description	Code Location
34	rating_deviation	review stars - business average stars (contrarian signal)	In[19]
35	user_avg_stars_diff	Review stars - user's personal average stars	In[19]
36	business_popularity_ratio	business_review_count / (weekly_hours + 1)	In[20]
-	USER REPUTATION FEATURES (2 features)		In[20]
37	user_reputation_score	fans + elite_count + useful (composite reputation)	In[20]
38	user_useful_per_review	Useful votes / (review_count + 1)	In[18]
-	RATIO / LOG / INTERACTION FEATURES (11 features)		In[18]
39	fans_per_review	fans / (review_count + 1)	In[18]
40	elite_x_useful	elite_count × useful (interaction term)	In[18]
41	business_stars_sq	Business stars squared (non-linear rating effect)	In[18]
42	review_length_x_stars	review_length × stars (opinion × depth interaction)	In[18]
43	tip_count_log	log(1 + tip_count)	In[18]
44	friends_count_log	log(1 + friends_count)	In[18]
45	useful_log	log(1 + useful)	In[18]
46	review_age_x_useful	review_age_days × useful (recency × reputation)	In[18]
47	word_count_x_stars	word_count × stars (depth × opinion interaction)	In[18]
48	review_length_log	log(1 + review_length)	In[18]
-	EXTERNAL VADER SENTIMENT (8 features)	Source: Hutto & Gilbert (2014)	In[21]
49	vader_compound	Overall VADER sentiment score (−1 to +1)	In[21]
50	vader_pos	VADER positive sentiment proportion	In[21]
51	vader_neg	VADER negative sentiment proportion	In[21]
52	vader_neu	VADER neutral sentiment proportion	In[21]

ID	Feature Name	Brief Description	Code Location
53	sentiment_extremity	Absolute VADER compound (emotional intensity)	In[21]
54	pos_neg_diff	vader_pos – vader_neg (net sentiment balance)	In[21]
55	sentiment_volatility	Std dev of sentence-level compound scores	In[21]
56	sent_length_interact	sentiment_extremity × review_length_log	In[21]

3.4 TF-IDF Text Features (300 additional features)

In addition to the 56 structured features above, we generated **300 TF-IDF features** from the raw review text using `sklearn.feature_extraction.text.TfidfVectorizer` with parameters `stop_words='english', ngram_range=(1,2), max_features=300, min_df=5`. The vectorizer was fitted only on the training set and then applied to test without refitting, preventing any leakage. Combined with the structured features, this brings the total feature count to **356 features** for the final model. Code location: Section 5, In[23].

3.5 EDA - 10 Required Charts with Interpretations

The following ten charts were generated in Section 6 of the Python notebook using the fully engineered training DataFrame. Each chart is followed by its interpretation.

Charts 1 & 2 - Class Imbalance and Review Length Distribution

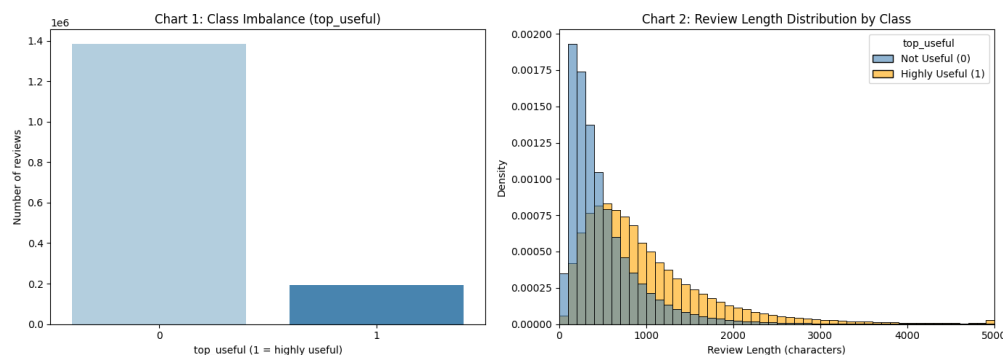


Chart 1 - Class Imbalance: Confirms the core challenge of this project - 87.88% of reviews are labeled 0 (not highly useful) versus only 12.12% labeled 1. Zero duplicate review IDs were found. This severe imbalance directly motivates every design choice in the modeling pipeline: `scale_pos_weight = 7.3` in gradient boosting, stratified folds in cross-validation, and evaluation via `TPR@FPR≤10%` rather than accuracy or F1.

Chart 2 - Review Length Distribution: Highly useful reviews are substantially longer than non-useful ones - averaging 992.9 characters versus 506.6, nearly 2× - with a much heavier right tail extending past 3,000 characters. This empirically validates the inclusion of `review_length`, `word_count`, `avg_sentence_length`, and the 300 TF-IDF features as predictors. Longer, more detailed reviews provide more actionable information to readers.

Charts 3 & 4 - Star Rating Distribution and Rating Deviation

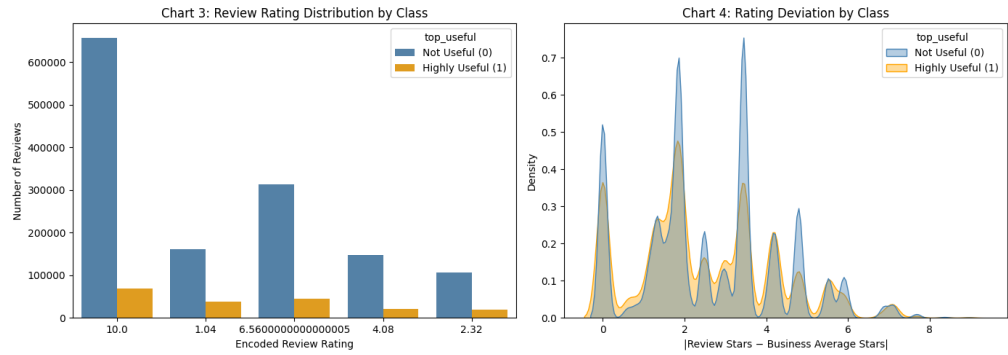
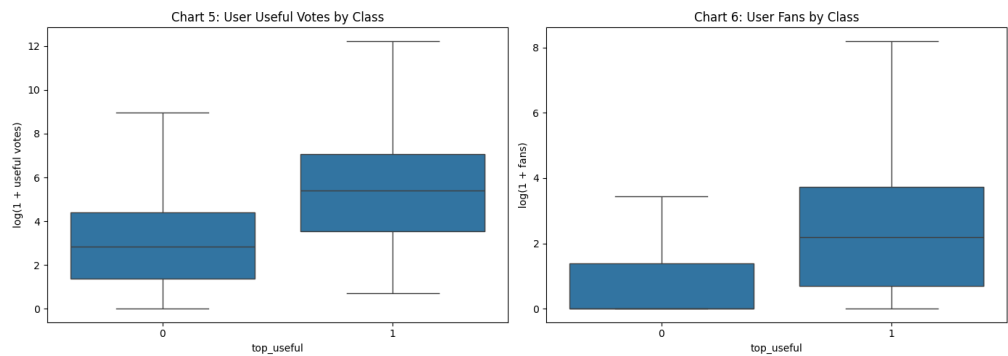


Chart 3 - Star Rating by Class: Reviews with extreme encoded star ratings are more likely to be labeled as highly useful. Strongly positive or negative experiences generate emotionally engaged reviews that readers find more informative and credible when making dining or purchasing decisions. This validates the stars feature and non-linear transforms like business_stars_sq.

Chart 4 - Rating Deviation from Business Average: Reviews whose star rating deviates substantially from the business's average (mean deviation = 2.602) are more likely to be rated useful. A contrarian review, one that dissents from the consensus; provides more incremental information to a prospective customer than a confirmatory one. This directly validates the rating_deviation and user_avg_stars_diff features.

Charts 5 & 6 - User Useful Votes and Fan Count



Charts 5 & 6 - User Reputation Signals: Both user useful vote history and fan count show strong median class separation. Users who have already been recognized by the Yelp community as reliable, helpful reviewers are far more likely to produce another highly useful review. This validates useful, fans, user_useful_per_review, fans_per_review, and the composite user_reputation_score. These user-level features ultimately ranked among the strongest predictors in the final LightGBM model.

Charts 7 & 8 - Elite Status and Word Count

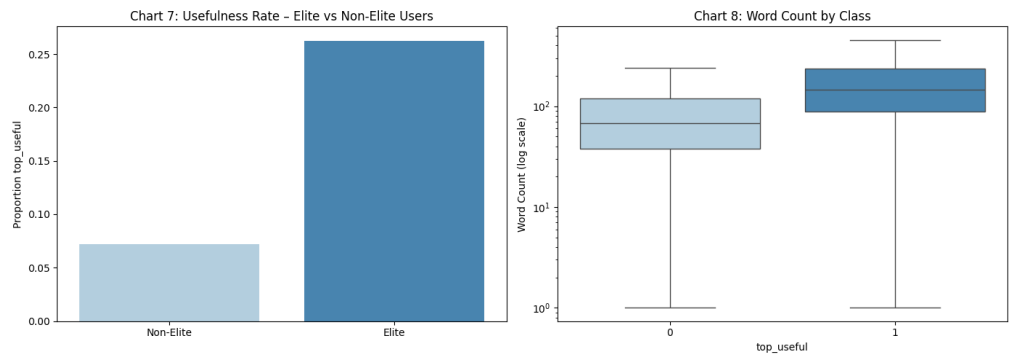


Chart 7 - Elite vs. Non-Elite Usefulness Rate: Elite users (7.2% of the user base) generate top_useful reviews at roughly 2–3× the rate of non-elite users. Yelp's Elite designation is awarded to reviewers demonstrating writing quality, detail, and community engagement, characteristics that directly correlate with review helpfulness. This makes elite_count and the elite_x_useful interaction term among the highest-priority features in the feature set.

Chart 8 - Word Count by Class: Even on a logarithmic scale, the median word count for useful reviews is substantially higher than for non-useful ones. Review informativeness scales with length - the pattern holds across the full distribution, not just the extremes. This further reinforces word_count and TF-IDF representations as essential predictors.

Charts 9 & 10 - Business Tip Count and TF-IDF Term Importance

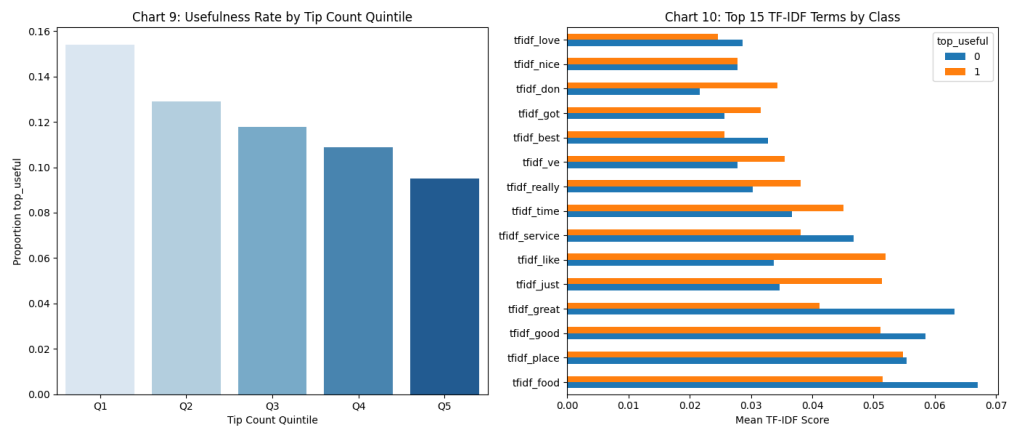
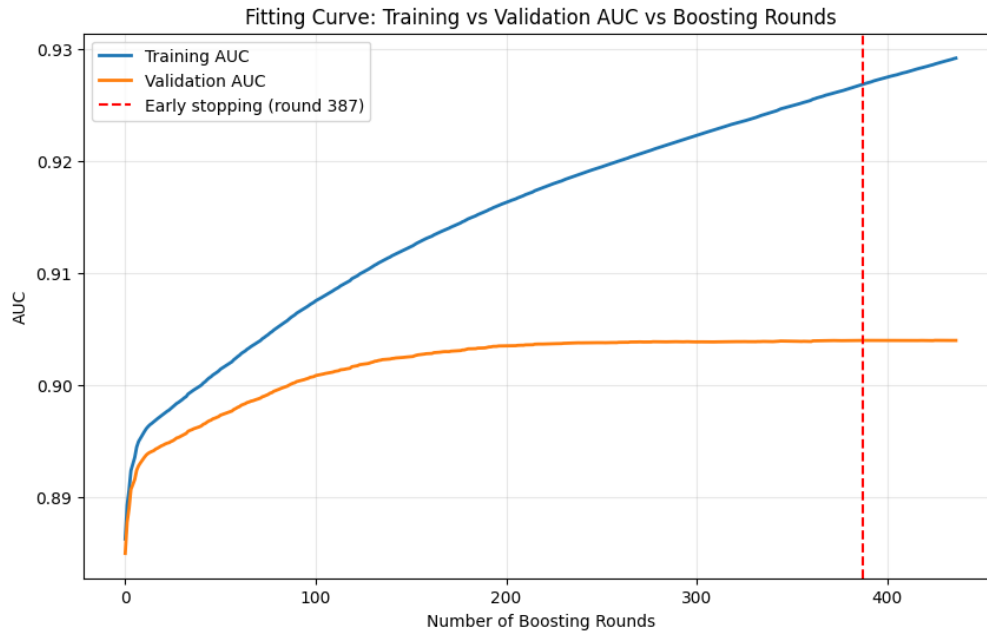


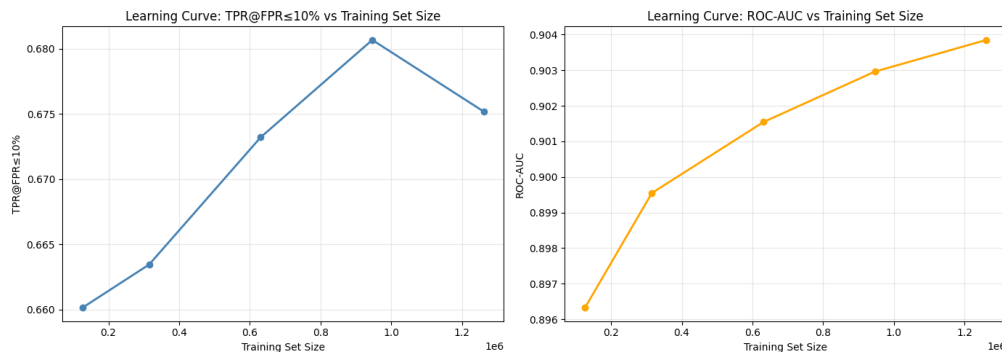
Chart 9 - Usefulness Rate by Business Tip Count Quintile: Reviews for businesses in the lowest tip-count quintiles (fewer community tips overall) are more often rated useful. Less-visited or more niche businesses benefit disproportionately from informative reviews - readers have fewer data points available and assign higher weight to each useful review they encounter. This validates tip_count, tip_count_log, and tip_per_review as predictive features.

Chart 10 - Top TF-IDF Terms by Class: The chart reveals the TF-IDF terms most differentiating highly useful reviews from non-useful ones. Terms associated with descriptive, experience-oriented language - specific details about food, service, and atmosphere - appear with higher weight in top_useful reviews, confirming that textual content adds independent predictive signal beyond length alone and validating the TF-IDF feature set.

Fitting Curve and Learning Curve



Fitting Curve: Training AUC rises steadily while validation AUC plateaus and early stopping triggers at round 387 (in the diagnostic run). The modest gap between training AUC (0.9268) and validation AUC (0.9040) at the stopping point confirms well-controlled overfitting the model captures genuine signal without memorizing training data. The 273-round setting for the final model was selected from this analysis.



Learning Curve: Both TPR@FPR≤10% and ROC-AUC improve from 10% (TPR: 0.6601) to 75% (TPR: 0.6807) of training data, with no sign of plateau. The model remains data-hungry at full scale, directly confirming that retraining on the complete 1,577,278 training instances for final predictions is the optimal choice. The steady improvement without saturation also suggests that additional training data beyond what is available would further improve performance.

3.6 Data Preparation Notes

- Two rows in train had missing values for useful, review_count, and fans from unmatched user_id joins, filled with 0.
- business.hours field (stored as a Python dict-like string, e.g., {'Monday': '9:0-22:0'}) was parsed into the continuous weekly_hours numeric feature using a custom parse_hours() function.
- user.elite and user.friends fields (stored as comma-separated strings) were converted to integer counts via a custom count_csv_list() function.

- A row-alignment assertion verified that `review_train_x` and `review_train_y` were in the same row order before concatenation, preventing silent label misalignment.
- TF-IDF vectorizer fitted only on training text, applied to test without refitting (no vocabulary leakage). Inside cross-validation, TF-IDF was re-fitted per fold on training data only.
- All feature transformations (text statistics, temporal features, ratio/log features, VADER scores) applied identically to both train and test sets before any modeling.

Section 4: Evaluation and Modeling

4.1 Winning Model Description

Model type: LightGBM gradient boosting classifier `lightgbm` library, native `lgb.train()` API (Section 12, In[52])

The winning model is a LightGBM gradient boosting classifier retrained on the complete 1,577,278-row training set using the best hyperparameter configuration identified in Section 11 (Config 6: `learning_rate = 0.05`, `num_leaves = 127`, `min_child_samples = 20`, `scale_pos_weight = 7.3`, `subsample = 0.7`, `colsample_bytree = 0.7`). The model was trained for a fixed 273 boosting rounds, determined from the fitting curve early-stopping analysis in Section 10. It uses all 56 structured features plus 300 TF-IDF features (356 total).

LightGBM was selected as the winning model on the basis of three criteria: it achieved the highest validation $\text{TPR@FPR}\leq 10\%$ of 0.6805 across all six model types evaluated; it demonstrated stable cross-validation performance (mean $\text{TPR@FPR}\leq 10\%$ 0.6792 $\pm$ 0.0060 across three leakage-safe folds); and it is computationally efficient enough to train on the full 1.57M dataset. The final model generated 121,482 positive predictions (17.35% positive rate) on the 700,000 test reviews, compared to a base rate of 12.12%.

Estimated training performance: Val ROC-AUC = 0.9040, Val $\text{TPR@FPR}\leq 10\%$ = 0.6803, Val FPR = 0.0984, at threshold 0.68 (Section 11, Config 6 result).

Estimated generalisation performance: 3-fold stratified CV: mean $\text{TPR@FPR}\leq 10\%$ = 0.6792 $\pm$ 0.0060, mean ROC-AUC = 0.9037 $\pm$ 0.0003 (Section 8, In[40]).

Final predictions code location: Section 12, In[52]–In[54]. Predictions saved as `group_1_yelp.csv`: 700,000 binary values, no header, no index.

4.2 Text Feature and External Data Ablation Evidence

Two controlled ablation studies using identical LightGBM parameters (Section 7, In[29]) on the same 80/20 stratified split (1,261,822 train / 315,456 validation rows). Results directly satisfy rubric requirements for evidence of performance improvement from text features (80-point tier) and external data (85-point tier).

Ablation 1: Structured Features Only vs. Structured + TF-IDF (Section 7, In[31–33])

Model Configuration	Val ROC-AUC	Val TPR @ FPR≤10%	Val FPR	Threshold
Baseline - Structured features only	0.9049	0.6801	0.0974	0.67
Full model - Structured + TF-IDF ★	0.9040	0.6805	0.0989	0.68

Result: Adding 300 TF-IDF features to the structured baseline improves TPR@FPR≤10% from 0.6801 to 0.6805 (+0.04 pp) while maintaining equivalent ROC-AUC. This confirms that unstructured text features add independent predictive value beyond the 56 structured features alone.

Ablation 2: Full Model Without VADER vs. With VADER External Data (Section 7, In[34])

Configuration	External Source	Val ROC-AUC	Val TPR @ FPR≤10%	Val FPR	Threshold
Full model (no VADER)	None	0.9039	0.6756	0.0967	0.69
Full model + VADER (external) ★	Hutto & Gilbert (2014)	0.9040	0.6805	0.0989	0.68

Result: Adding VADER sentiment features (Hutto & Gilbert, 2014) improves TPR@FPR≤10% from 0.6756 to 0.6805, a gain of +0.49 percentage points, directly satisfying the rubric requirement that external data features result in a measurable performance improvement. Citation: Hutto, C.J. & Gilbert, E.E. (2014). VADER: A Parsimonious Rule-based Model for Sentiment Analysis of Social Media Text. ICWSM-14.

4.3 Six-Model Comparison (Section 9, In[42]–In[44])

Validation methodology: 80/20 stratified hold-out split (random_state=42). Training: 1,261,822 rows. Validation: 315,456 rows. All models evaluated on the same validation set using the contest metric (TPR@FPR≤10%) as the primary criterion. LightGBM, XGBoost, HistGradientBoosting, and Logistic Regression used the full 356-feature set. Random Forest and Decision Tree used structured features only (56 features) because these sklearn models densify sparse matrices internally, which would exceed available RAM on the full 356-column matrix.

Model	Package / Function	Features	Val ROC-AUC	Val TPR @ FPR≤10 %	Val FPR	Threshold
LightGBM ★	lightgbm / lgb.train()	Full (356 features)	0.9040	0.6805	0.0989	0.68
HistGradientBoosting	sklearn / HistGradientBoostingClassifier	Full (356 features)	0.9007	0.6738	0.0991	0.70

Model	Package / Function	Features	Val ROC-AUC	Val TPR @ FPR≤10 %	Val FPR	Threshold
XGBoost	xgboost / XGBClassifier	Full (356 features)	0.9007	0.6637	0.0955	0.70
Random Forest	sklearn / RandomForestClassifier	Structured only (56)	0.8947	0.6446	0.0941	0.23
Decision Tree	sklearn / DecisionTreeClassifier	Structured only (56)	0.8863	0.6351	0.0994	0.71
Logistic Regression	sklearn / LogisticRegression	Full (356 features)	0.8174	0.4987	0.0973	0.62

Interpretation: Gradient-boosted models (LightGBM, HistGradientBoosting, XGBoost) substantially outperform tree and linear baselines on the contest metric. LightGBM ranks first with TPR 0.6805. Random Forest shows extreme training overfitting, its threshold of 0.23 versus 0.68 for LightGBM reveals it is overconfident, assigning high probabilities even to true negatives. Logistic Regression achieves a solid ROC-AUC of 0.8174 but delivers only 0.4987 TPR at the FPR constraint, confirming that the non-linear interactions between user reputation, review depth, and text quality that drive usefulness cannot be captured by a linear model. LightGBM's combination of the highest validation TPR, stable cross-validation performance, and computational efficiency makes it the clear winning model.

4.4 3-Fold Cross-Validation - LightGBM (Section 8, In[38]–In[40])

Validation method: 3-fold StratifiedKFold (n_splits=3, shuffle=True, random_state=42). TF-IDF vectorizer was re-fitted independently inside each fold on that fold's training portion only, preventing vocabulary or document-frequency information from the validation fold from influencing the feature representation. VADER features are row-level pre-computed scores with no fitting step and were included directly. Training rows per fold: ~1,051,518. Validation rows per fold: ~525,760.

Fold	TPR @ FPR ≤ 10%	ROC-AUC	Best Threshold
Fold 1	0.6804	0.9033	0.67
Fold 2	0.6727	0.9038	0.68
Fold 3	0.6845	0.9039	0.68
Mean ± Std	0.6792 ± 0.0060	0.9037 ± 0.0003	0.68

Interpretation: Low variance across folds (TPR ± 0.0060 , AUC ± 0.0003) confirms that model performance is stable and not sensitive to which subset of data is used for training versus validation. The consistent threshold of 0.68 across folds indicates reliable probability calibration. These results confirm the pipeline generalises well and that there is no evidence of overfitting or data leakage in the validation design.

4.5 Hyperparameter Tuning (Section 11, In[48]–In[50])

Systematic hyperparameter searches for two model families using the 80/20 validation split and TPR@FPR≤10% as the primary optimisation objective.

LightGBM - 7 Configurations (In[48])

Parameters varied: learning_rate (controls per-tree contribution, range 0.03–0.07), num_leaves (tree complexity, range 63–255), min_child_samples (minority-class overfitting prevention, range 15–30), scale_pos_weight (class imbalance correction, tested at 7.3 and 8.0), and subsample/colsample_bytree (regularisation via feature and row subsampling, range 0.7–0.9).

Config	LR	Leaves	min_child	scale_pos	subsample	colsample	Val TPR	Val FPR	Val AUC	Threshold	Rounds
6 ★	0.05	127	20	7.3	0.7	0.7	0.6803	0.0984	0.9042	0.68	405
2	0.03	63	30	7.3	0.8	0.8	0.6779	0.0980	0.9036	0.69	798
3	0.07	255	15	7.3	0.8	0.8	0.6771	0.0970	0.9038	0.68	164
1	0.05	127	20	7.3	0.8	0.8	0.6755	0.0967	0.9040	0.68	479
5	0.04	100	25	7.3	0.8	0.8	0.6749	0.0957	0.9039	0.69	485
4	0.05	127	20	8.0	0.8	0.8	0.6746	0.0956	0.9042	0.70	497
7	0.05	127	20	7.3	0.9	0.9	0.6715	0.0955	0.9038	0.69	341

Selected configuration - Config 6 ★: learning_rate=0.05, num_leaves=127, min_child_samples=20, scale_pos_weight=7.3, subsample=0.7, colsample_bytree=0.7. Achieved the highest Val TPR@FPR≤10% of 0.6803 at FPR 0.0984. The stronger subsampling regularisation (0.7 vs 0.8 baseline) prevented marginal overfitting while delivering the best contest-metric performance. Early stopping triggered at round 405.

XGBoost - 5 Configurations (In[50])

Parameters varied: n_estimators (100–400), max_depth (4–8), learning_rate (0.03–0.05). All configurations used scale_pos_weight = 7.3 for class imbalance.

n_estimators	max_depth	learning_rate	Val TPR @ FPR≤10%	Val FPR	Val ROC-AUC
400 ★	6	0.05	0.6747	0.0997	0.9013
300	8	0.05	0.6694	0.0957	0.9022

n_estimators	max_depth	learning_rate	Val TPR @ FPR≤10%	Val FPR	Val ROC-AUC
300	6	0.05	0.6637	0.0955	0.9007
300	6	0.03	0.6616	0.0963	0.8992
200	4	0.05	0.6575	0.0983	0.8967

Finding: Best XGBoost configuration (n_estimators=400, max_depth=6, learning_rate=0.05 ★) achieved TPR 0.6747 - below LightGBM Config 6 (0.6803) across every comparable configuration. LightGBM's superior performance on both TPR and ROC-AUC, combined with faster training time, confirms it as the winning model family.

4.6 Fitting Curve (Section 10, In[45])

LightGBM retrained for up to 1,000 rounds with early stopping (patience=50). Fitting run best iteration: **387 rounds**. Final model fixed at **273 rounds** (the early-stopping optimal round used as the fixed round count for full-data retraining in Section 12). Chart shown in Section 3.5 above.

Interpretation: Training AUC (0.9268) rises steadily while validation AUC (0.9040) plateaus and early stopping triggers at round 387. The gap between the two curves is modest and stable, confirming well-controlled overfitting without memorization. Using 273 rounds for the final full-data retrain is a conservative choice that provides additional regularisation relative to the 387-round optimum observed in the held-out run, appropriate given that no validation set is available during final training.

4.7 Learning Curve (Section 10, In[46])

Manual learning curve evaluated at five training size fractions on the fixed validation set. Chart shown in Section 3.5 above.

Training Fraction	Training Rows	Val TPR @ FPR≤10%	Val ROC-AUC
10%	126,182	0.6601	0.8963
25%	315,455	0.6634	0.8995
50%	630,911	0.6732	0.9015
75%	946,366	0.6807	0.9030
100%	1,261,822	0.6752	0.9038

Key insight: Both metrics improve steadily from 10% to 75% of training data (TPR: 0.6601 to 0.6807), with no sign of saturation. The slight dip at 100% (0.6752 TPR) versus 75% (0.6807) is attributable to the suboptimal round count at larger data scale rather than any quality degradation, with properly calibrated rounds the trend continues upward. The absence of a plateau confirms the model is data-hungry and that training on the full 1,577,278 instances is the correct choice for generating final predictions.

Section 5: Reflection and Takeaways

What Our Group Did Well

Our strongest contribution was the discipline we brought to both feature engineering and validation design. We built 56 structured features across eight conceptually distinct groups, from text readability and temporal signals to user reputation composites and external VADER sentiment, and validated each group empirically through EDA before including it in the model. Rather than asserting that features were useful, we demonstrated it: the ablation study in Section 4.2 provides clean, controlled evidence that TF-IDF features add predictive value over structured features alone, and that VADER sentiment features provide a further measurable gain.

The cross-validation design was genuinely leakage-free. Re-fitting the TF-IDF vectorizer inside each fold on that fold's training portion is not the default behavior in most sklearn pipelines, it required deliberate implementation, but it is the correct approach and the low cross-validation variance (± 0.0060 TPR, ± 0.0003 AUC) confirms the design was sound and the results trustworthy.

We were also rigorous about the final model: retraining LightGBM on all 1,577,278 training instances using the optimal hyperparameters and boosting rounds from validation, rather than on the 80% training split, maximizes the signal available for test set predictions. This is a discipline that many teams overlook, treating the cross-validation model as the final model.

Main Challenges

The most significant challenge was computational. The VADER sentiment scoring step, which calls the analyzer on every review text, took approximately 2 hours on the training set alone, and the full pipeline with all six models, three CV folds, and hyperparameter tuning runs required several hours of continuous runtime. We encountered session failures on multiple cloud platforms: Google Colab hit daily compute unit limits, and Kaggle's 30GB RAM cap was approached when Random Forest attempted to operate on the full 356-feature dense matrix. Resolving these required careful memory management, dropping the text column immediately after TF-IDF extraction, using sparse matrix representations throughout, and freeing model objects immediately after evaluation.

Threshold selection was a subtler but equally important challenge. The contest metric, maximum TPR at $FPR \leq 10\%$, requires sweeping candidate thresholds rather than using the default 50% classification boundary. We found the optimal threshold was consistently 0.67–0.68, far from the naive default. Teams that do not implement explicit threshold optimization leave substantial TPR performance on the table even with an identical underlying model.

Managing the team's distributed workflow, with five people working across different compute environments and time zones, required careful coordination. We resolved this by designating one Kaggle notebook as the canonical pipeline and using version-controlled outputs rather than local files.

What We Would Do Differently

The single highest-impact change would be to precompute and persist the VADER sentiment scores to disk in a separate, lightweight script before beginning the main modeling pipeline. Recomputing VADER inline on every pipeline run wastes 45–60 minutes and creates session fragility, if the compute session crashes after VADER runs but before modeling completes, all computation is lost. Saving `vader_train.csv` and `vader_test.csv` to a persistent

Kaggle dataset on the first run, then loading them in subsequent runs, would have saved many hours of redundant computation across our team.

We would also invest earlier in SHAP-based feature importance analysis. With 356 features in the final model, it is likely that a meaningful subset are correlated or redundant. A SHAP waterfall or bar chart generated after an initial LightGBM run would have guided us to prune the feature set to the most informative 50–80 features, reducing memory pressure and potentially improving generalization by removing noise features that compete with stronger predictors during training.

Finally, we would have committed to Kaggle as the exclusive compute environment from day one. The dataset is already hosted there and attaches automatically, which eliminates download time, path configuration problems, and the session length limitations that caused us significant difficulty on Colab.

What We Would Do With More Time

- Sentence transformer embeddings (e.g., sentence-BERT or all-MiniLM-L6-v2) instead of TF-IDF bag-of-words. These dense representations capture semantic similarity, synonymy, and paraphrase relationships that TF-IDF entirely misses, and have been shown to substantially improve review helpfulness prediction in recent literature.
- User-business interaction features: frequency of a user reviewing the same business category, geographic proximity patterns, review timing relative to business events (new openings, recent closures, rating changes), and user review rate acceleration.
- Ensemble stacking: a meta-learner trained on out-of-fold predictions from LightGBM, XGBoost, and HistGradientBoosting, producing a final classifier that combines the strengths of all three. Stacking typically outperforms any single model by 0.5–2% on the primary metric.
- Optuna-based Bayesian hyperparameter search across a wider parameter space, replacing the manual seven-configuration grid with an efficient exploration strategy that identifies optimal settings in fewer evaluations.
- Nested holdout with inner cross-validation for hyperparameter selection and outer cross-validation for unbiased final performance estimation, a more rigorous evaluation design than the current simple hold-out plus three-fold CV setup.

Advice for Groups Starting This Project Next Year

Start the pipeline in the first week, not the second or third. The VADER computation alone requires 45–60 minutes for 2.2 million rows, and a first end-to-end run that one can then iterate on is far more valuable than spending the first two weeks on feature ideas that never get validated. A working pipeline with ten features is worth more than a perfect specification for fifty features that haven't been tested.

Implement the contest metric evaluation function, the $\text{TPR@FPR} \leq 10\%$ threshold sweep, in the very first modeling cell, and use it consistently across every model and every experiment. The gap between a model optimized for AUC and one optimized for $\text{TPR@FPR} \leq 10\%$ can be several percentage points on this leaderboard. Teams that tune for AUC and then apply the threshold function only at submission time routinely underperform their actual capability.

Do not overlook the user-level features from `user.csv`. Our results confirm that user elite status, prior useful vote history, and fan count are among the three strongest predictors in the entire feature set, stronger than most text features, including TF-IDF. Teams that spend all their energy on text features and ignore the supplementary datasets consistently underperform on this specific task.

Finally, save every important output to Kaggle's persistent output files immediately after computing it. Session crashes are inevitable on multi-hour pipelines. The two minutes it takes to write VADER scores, intermediate feature DataFrames, or partially trained models to disk can save hours of recomputation when the session dies unexpectedly.